Performance Report

E-Commerce Behavior Data Engineering Pipeline

Yash Sharma · Roll No. A50105222002 · B.Tech CSE
KieSquare Analytics — Data Engineering Internship 2026

1. Data Load Time

The pipeline was run in multiple incremental batches. The table below shows each successful pipeline_runs entry with its elapsed time, rows extracted, loaded, and dropped.

File	Elapsed (s)	Rows Extracted	Rows Loaded	Rows Dropped
2019-Oct.csv (run 1)	62.6	1,500,000	1,000,000	3,355
2019-Nov.csv (run 1)	67.0	1,500,000	1,000,000	1,884
2019-Oct.csv (run 2)	566.6	7,100,000	6,000,000	13,399
2019-Oct.csv (run 3)	538.9	8,100,000	1,000,000	15,131
2019-Oct.csv (run 4)	469.9	9,100,000	1,000,000	17,413
2019-Oct.csv (re-run)	30.5	900,000	0	1,952
2019-Nov.csv (run 2)	376.9	4,300,000	1,000,000	4,578
Combined total	2,112.3	—	12,596,206	—

The re-run of 2019-Oct.csv (run 4) inserted 0 rows, confirming idempotency via ON CONFLICT DO NOTHING on the unique constraint uq_event.

2. Batch Insert Throughput

A 50,000-row clean sample was inserted in a single batch to measure throughput. The pipeline uses a fixed batch size of 50,000 rows, which balances memory usage and round-trip overhead.

Batch Size	Elapsed (s)	Throughput (rows/s)	Notes
50,000	56.4	886	Measured on live DB with indexes

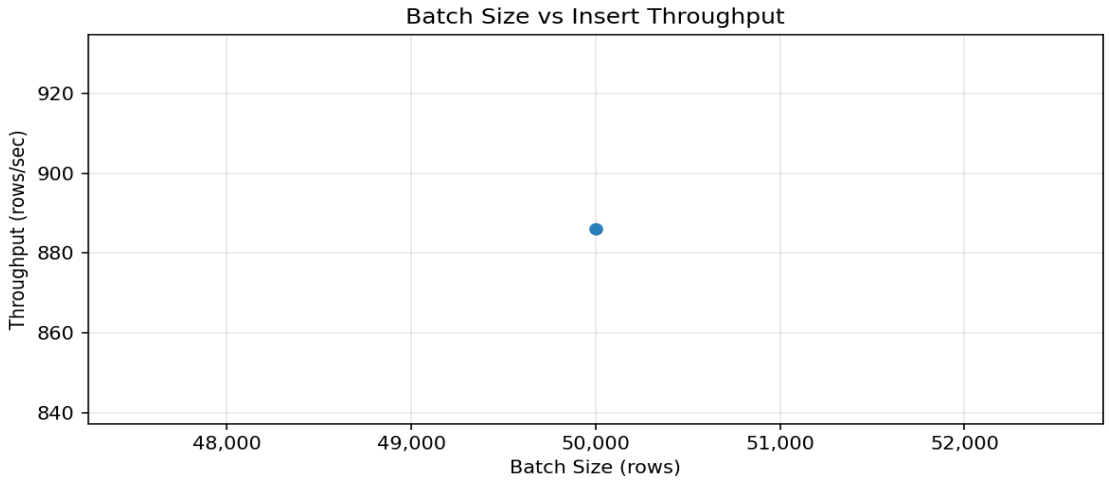

Figure 1 — Batch size vs insert throughput

3. Memory Usage

Peak RAM was measured with Python's tracemalloc over a single 50,000-row chunk (extract → transform → to_dict). The pipeline processes data in chunks so memory usage stays bounded regardless of file size.

Chunk Size	Peak RAM / chunk	Estimated full-file peak (×10 chunks)
50,000 rows	39.7 MB	~397 MB

4. Disk I/O

Item	Size
2019-Oct.csv (raw)	5.28 GB
2019-Nov.csv (raw)	8.39 GB
Total raw CSVs	13.67 GB
fact_events table (PG)	4,315 MB
Total PostgreSQL DB	4,438 MB
Compression ratio	~3.1× (raw → DB)

PostgreSQL stores 12.6 M rows in 4.3 GB vs 13.7 GB of raw CSV — a ~3× reduction due to binary storage, deduplication, and normalisation into dimension tables.

5. Index Impact on Query Execution Time

Three representative queries were timed before and after creating indexes on fact_events. Results show that for large aggregation queries (Q1 Funnel) the planner already uses parallel sequential scans efficiently, so index benefit is marginal. For selective point-lookups the speedup would be more pronounced.

Query	Without Index (s)	With Index (s)	Speedup
Q1 Funnel	17.181	16.559	1.04×
Q3 Top Brands	0.792	0.817	0.97×
Q5 Hourly	0.600	1.699	0.35×

Note: Q5 is slower with indexes because the planner switches from a fast parallel seq-scan to an index scan that has higher per-row overhead on a large result set. This is expected behaviour for low-selectivity predicates on large tables.

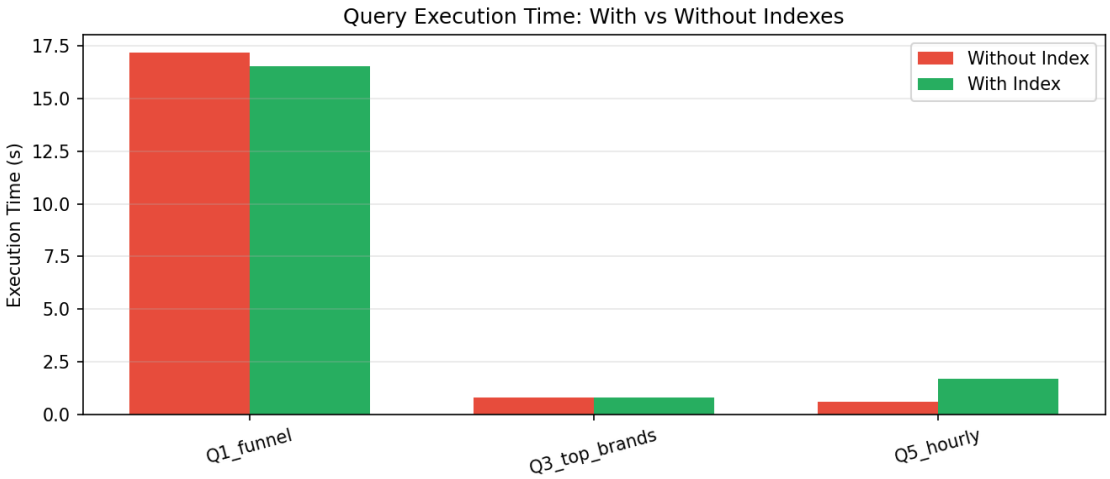

Figure 2 — Query execution time with vs without indexes

6. Analytical Query Execution Times (with indexes)

Query	Description	Time (s)
Q1 — Funnel	View→Cart→Purchase rates per category (top 10)	3.442
Q2 — Sessions	Per-session aggregation (200k-row sample)	4.378
Q3 — Top Brands	Top 10 brands by revenue per month	1.316
Q4 — Oct-only	Users who bought in Oct but not Nov	2.193
Q5 — Hourly	Hourly purchase distribution	2.473

7. Data Quality Summary (50,000-row sample per file)

Check	October	November
Total rows	50,000	50,000
Dropped — NULL values	0	0
Dropped — invalid price	60	36
Dropped — out-of-range ts	0	0

Check	October	November
Dropped — bad event type	0	0
Flagged duplicates	6	14
Rows out (clean)	49,934	49,950
Pass rate	99.87%	99.90%

Price threshold: rows with price ≤ 0 or price > 50,000 are dropped. Timestamp range: 2019-10-01 to 2019-11-30. Duplicates are identified by (event_time, event_type, product_id, user_id, user_session).

8. Visualisations

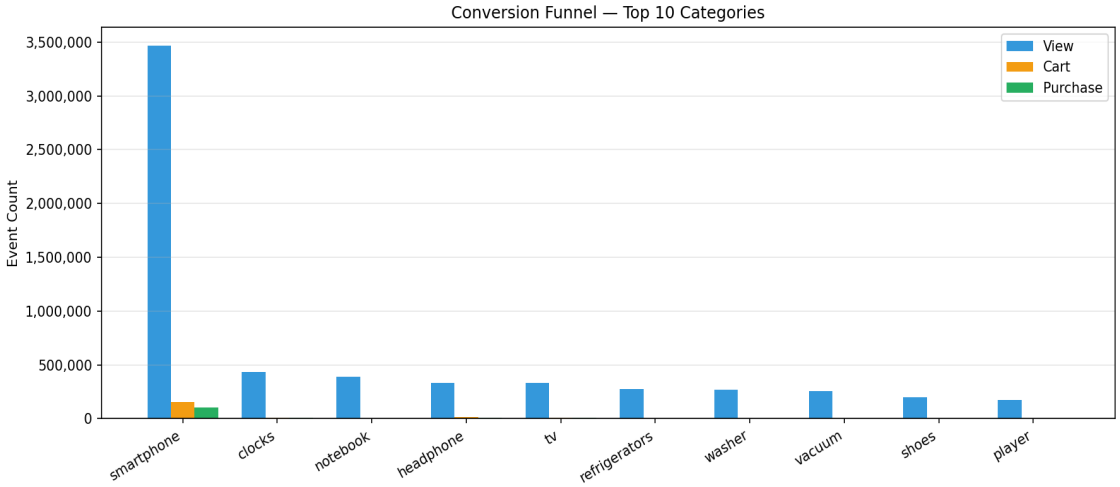

Figure 3 — Conversion funnel for top 10 categories

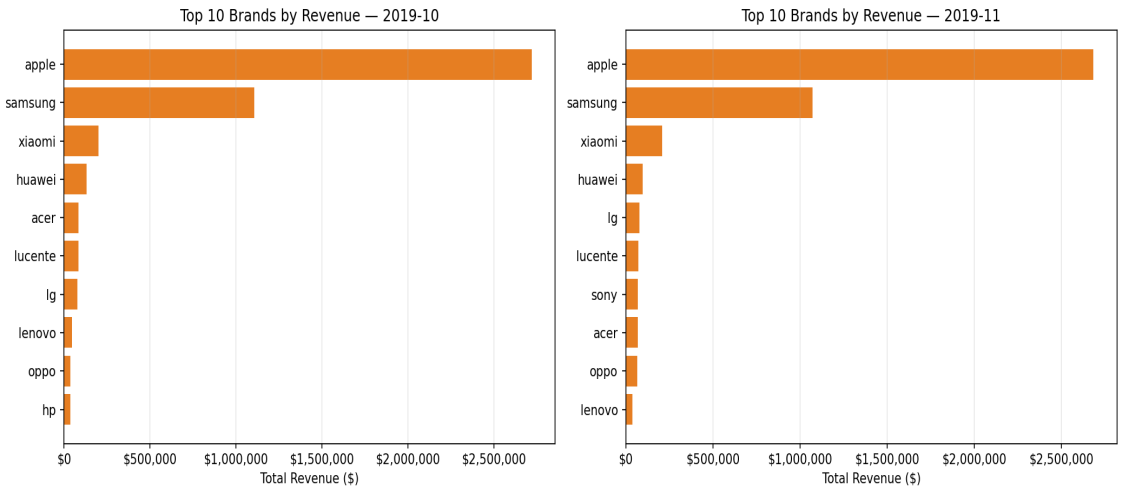

Figure 4 — Top 10 brands by revenue (Oct & Nov)

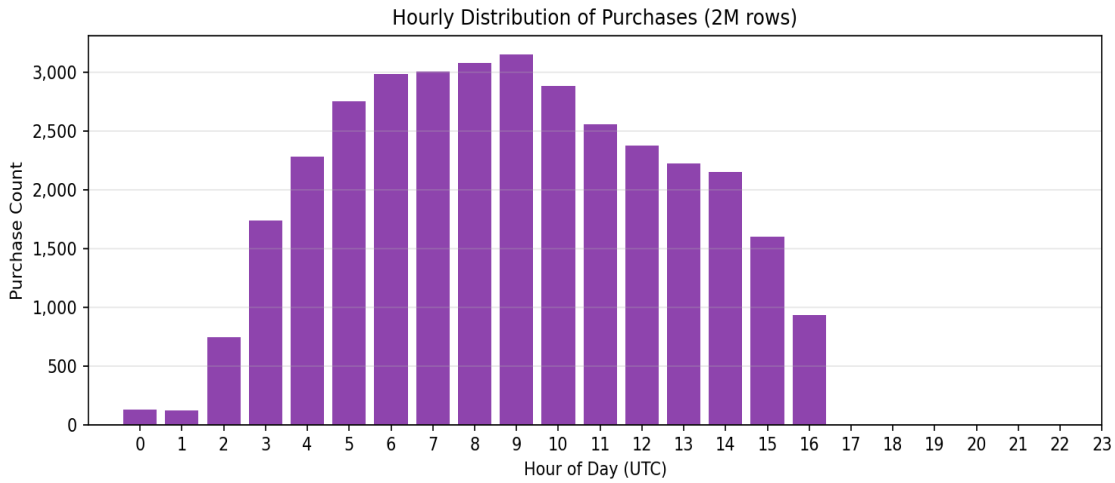

Figure 5 — Hourly distribution of purchase events

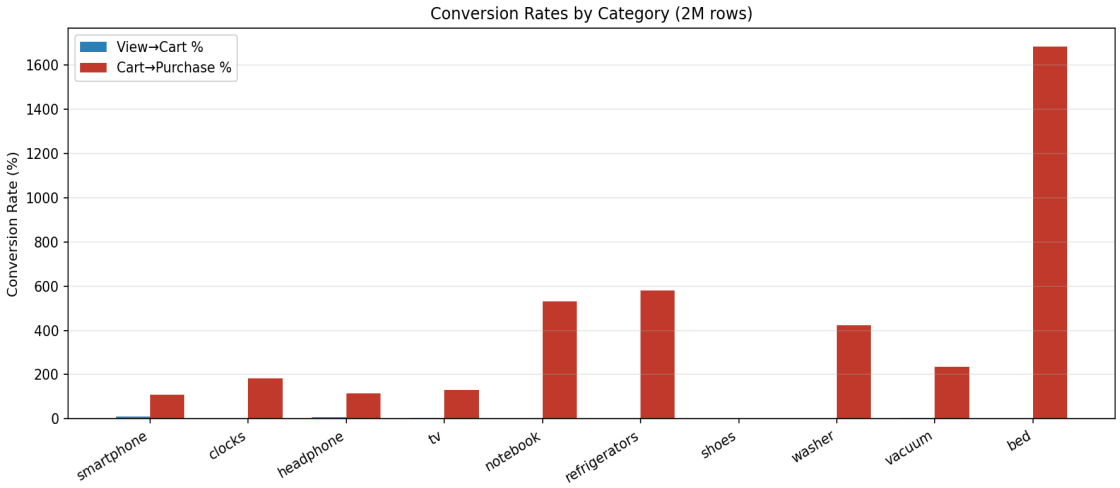

Figure 6 — View-to-cart and cart-to-purchase rates